



LAB EXERCISES
Distributed Algorithms (IN4150)

Distributed Systems (DS)
Department Software Technology
Faculty EEMCS

Assignment 2

RELIABLE BROADCAST

2024-2025

Assignment

In this lab assignment, you will implement Bracha’s reliable broadcast algorithm [1, 2]. The algorithm is designed to work in a network where up to f of the N processes can be Byzantine. Please ensure that the solution adheres to the following requirements:

1. The algorithm is implemented using the Python template and the IPv8 networking library (available on Brightspace).
2. The implementation runs across multiple Docker containers.
3. The network structure and the node behaviour are adjustable (e.g. network size and messages sent).
4. Each node incorporates random delays before sending a message in order to emulate network conditions (e.g. through traffic control).
5. Each node logs events in a **human-readable** fashion (e.g. to a log file or terminal).
6. The code is submitted to the GitLab repository.

The assignment can be split up into three parts:

Reliable Broadcast: Bracha’s algorithm

We aim to implement a distributed algorithm that enforces the Reliable Broadcast (BRB) abstraction on a **(virtual) fully connected network**. BRB requires the following properties:

BRB-Validity If a correct process p broadcasts a message m , then every correct process eventually BRB-delivers m .

BRB-No duplication No correct process BRB-delivers a message m more than once.

BRB-Integrity If a correct process BRB-delivers a message m with sender p_i , then m was previously broadcast by p_i .

BRB-Agreement If some correct process BRB-delivers a message m , then every correct process eventually BRB-delivers m .

Part A: Byzantine Reliable Broadcast

Bracha and Toueg formalized the Reliable Broadcast problem [1, 2] and then Bracha described the first BRB protocol for asynchronous and fully connected reliable networks (i.e., networks where each process is able to communicate with any other in the system and where messages cannot be lost) and the proposed solution is able to tolerate f Byzantine nodes (where $f < N/3$ and N is the

number of processes in the system) [3]. This protocol is characterized by three different all-to-all communication phases (namely, *send*, *echo* and *ready*) and processes progress in the algorithm as soon as they have heard from a quorum of nodes in a given phase.

The protocol works in three phases. When a process p_i wants to BRB-broadcast some payload data, it sends a **SEND** message along with the payload data to all nodes in the system (phase 1). Upon receiving a **SEND** message, a process sends an **ECHO** message to all the nodes in the system (along with the payload data) and moves to phase 2 where it remains waiting for a quorum of **ECHO** messages. Upon receiving $\left\lceil \frac{N+f+1}{2} \right\rceil$ **ECHO** messages for a given payload, a process sends a **READY** message to other nodes and moves to phase 3. Let us note that a **READY** message can also be sent upon receiving $f+1$ **READY** messages as it will ensure that at least one correct process is moving to phase 3 and thus it is safe to broadcast a **READY** message and to move to phase 3 as well. Finally, upon receiving $2f+1$ **READY** messages, a process can BRB-deliver the payload data by concluding the procedure.

For the sake of completeness, we report in Algorithm 1 the pseudo-code of the algorithm, which we extract from [4, 5].

Implement Bracha's algorithm in a non-fully connected network by having it **explicitly** rely on Dolev's algorithm (cf. Assignment 1). Intuitively, Dolev's algorithm will provide the illusion that pairs of nodes are connected, which is required by Bracha's algorithm.

Part B: Optimizations

Implement the following optimizations that do not appear in the pseudocode. Please note that the 'Single-hop Send' optimization requires the 'Echo amplification' to work correctly.

Echo amplifications. In Bracha's protocol, it is known (and necessary) that the reception of $f+1$ Readys allows a process to generate its own Ready message, if it has not done so already. This situation can happen because the links are asynchronous. However, the delivery of $f+1$ Echos should also allow a process to generate its own Echo message and send it if it was not sent before. To accelerate the creation of a Ready message, one can also observe that delivering a Ready message allows the immediate creation and delivery of the associated Echo message. It might also happen that the Dolev-delivery of a Ready message allows a process to generate both its own Echo and its own Ready message. In such a case, the process only sends a Ready message.

Single-hop Send messages [MBD.2]. In the default combination of Bracha's and Dolev's protocols, the Send messages that a correct process creates reach all the processes. We modify the protocol so that upon receiving a Send message, a neighbor of the source stops disseminating it, and creates instead an Echo message that it relays to all its neighbors. When a process

Algorithm 1 *BRB in asynchronous and fully connected networks (Bracha's protocol) at process p_i*

```

1: Parameters:
2:    $\Pi$ : the set of all processes.
3:    $N$ : total number of processes.
4:    $f < N/3$ : maximum number of Byzantine processes.
5: Uses: Auth. async. perfect point-to-point links, instance  $al$ .
6:
7: upon event  $\langle Bracha, Init \rangle$  do
8:    $sentEcho = sentReady = delivered = \mathbf{False}$ 
9:    $echos = readys = \emptyset$ 
10:
11: upon event  $\langle Bracha, Broadcast \mid m \rangle$  do
12:   forall  $q \in \Pi$  do { trigger  $\langle al, Send \mid q, [SEND, m] \rangle$  }
13:
14: upon event  $\langle al, Deliver \mid p, [SEND, m] \rangle$  and not  $sentEcho$  do
15:    $sentEcho = \mathbf{True}$ 
16:   forall  $q \in \Pi$  do { trigger  $\langle al, Send \mid q, [ECHO, m] \rangle$  }
17:
18: upon event  $\langle al, Deliver \mid p, [ECHO, m] \rangle$  do
19:    $echos.insert(p)$ 
20:
21: upon event  $echos.size() \geq \lceil \frac{N+f+1}{2} \rceil$  and not  $sentReady$  do
22:    $sentReady = \mathbf{True}$ 
23:   forall  $q \in \Pi$  do { trigger  $\langle al, Send \mid q, [READY, m] \rangle$  }
24:
25: upon event  $\langle al, Deliver \mid p, [READY, m] \rangle$  do
26:    $readys.insert(p)$ 
27:
28: upon event  $readys.size() \geq f+1$  and not  $sentReady$  do
29:    $sentReady = \mathbf{True}$ 
30:   forall  $q \in \Pi$  do { trigger  $\langle al, Send \mid q, [READY, m] \rangle$  }
31:
32: upon event  $readys.size() \geq 2f+1$  and not  $delivered$  do
33:    $delivered = \mathbf{True}$ 
34:   trigger  $\langle Bracha, Deliver \mid s, m \rangle$ 

```

p_i Dolev-delivers a $[SEND, (s, bid), m, path]$ message that it received from a neighbor p_j , the combination of Bracha's and Dolev's algorithms makes it forward a modified Send message to its neighbors not included in path, i.e., $[SEND, (s, bid), m, path + [p_j]]$. A process can avoid generating and forwarding the modified Send message without loss of information, as it can be extracted from Echo messages. A Send message is therefore a single-hop message and does not need to carry a path. More generally, note that an Echo message from the source can be extracted from any message that a process receives, which might also enable a node to send its own Echo or Ready faster.

Reduced number of messages in Bracha [MBD.11]. To implement this method, we assume that processes know the ID of all processes in the system and that $N > 3f + 1$ [6]. When process p_i BD-broadcasts a message, the processes with the $\lceil \frac{N+f+1}{2} \rceil + f$ smallest IDs after p_i 's (modulo the total number

of processes) generate Echos, while the processes with the $2f+1+f$ smallest IDs generate Readys. The other processes simply relay the messages they receive. All processes deliver when they have collected $2f+1$ Readys. Note that the processes that participate actively in a broadcast, i.e., generate new Echo or Ready messages, depend on the ID of the process that initiated the broadcast to distribute the load over all processes.

Part C: Testing & Reporting

Do not forget to evaluate your code.

For the sign-off, **be ready to demonstrate** that:

- A correct node can broadcast several messages and get them all delivered.
- A faulty node that initiates a broadcast and sends its message to a limited number of neighbors either leads all correct nodes to deliver it, or none at all.
- Even if all faulty nodes collude they do not lead any correct node to deliver a forged message from a correct node.

Report

In this report (**PDF** format), you will present your implementation and benchmarking of a distributed algorithm with minimal text. Focus on presenting clear metrics and plots that demonstrate your algorithm's performance under different conditions. Include at least the following.

1. Title page

- List all group members and the algorithm implemented.
- Briefly describe the purpose of the algorithm (max. 4 sentences).

2. Experimental Setup

Summarize the key parameters you used in a **table** (e.g., number of nodes, frequency of message drops, failure rate).

3. Results

For each metric, provide a brief description (few sentences) followed by the relevant plots. Ensure plots are clear and labeled, with axis labels and a title. Use consistent units across plots (e.g., seconds for time, messages/sec for throughput). Minimum set of plots to provide:

- Latency and message complexity without or with the 'Echo amplifications' optimization.

- Latency and message complexity without or with the 'Single-hop Send messages' and the 'Echo amplifications' optimizations.
- Latency and message complexity without or with optimization MBD.11.

4. Conclusion

Summarize the key findings from your plots. For example: “Our implementation scales well up to 10 nodes, but latency significantly increases with larger networks.”.

Tips

- Avoid lengthy descriptions or text blocks; use bullet points or tables where possible.
- Focus on clear, high-quality visuals that allow for quick interpretation.
- Use plot types appropriate for the metric (e.g., line, box, bin).
- Use a sufficient number of points for your plots, e.g., 10 for a line.

References

- [1] G. Bracha, “An asynchronous $[(n-1)/3]$ -resilient consensus protocol,” in *PODC*, 1984.
- [2] G. Bracha and S. Toueg, “Asynchronous consensus and broadcast protocols,” *J. ACM*, vol. 32, no. 4, pp. 824–840, 1985.
- [3] G. Bracha, “Asynchronous byzantine agreement protocols,” *Inf. Comput.*, vol. 75, no. 2, pp. 130–143, 1987.
- [4] C. Cachin, R. Guerraoui, and L. E. T. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*. 2011.
- [5] M. Raynal, *Fault-tolerant message-passing distributed systems: an algorithmic approach*. 2018.
- [6] S. Bonomi, J. Decouchant, G. Farina, V. Rahli, and S. Tixeuil, “Practical byzantine reliable broadcast on partially connected networks,” in *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*, pp. 506–516, IEEE, 2021.